

Agency and Package Availability Management System

Scenario

The Travel Booking Management System manages travel agencies, travel packages, and package availability. Multiple travel agencies can provide different travel packages. The system maintains agency details, connects each package with its agency, and monitors the number of available slots for each package.

The purpose of Week 3 is to create the TravelAgency and PackageAvailability tables, establish relationships with the existing TravelPackage table, apply constraints, insert records, and generate agency-wise and availability-based reports.

Create TravelAgency and PackageAvailability Tables

TravelAgency Table

The CREATE TABLE command creates the TravelAgency table. It stores details about the travel agencies that provide travel packages.

```
SQL>
SQL> CREATE TABLE TravelAgency (
  2   Agency_ID NUMBER,
  3   Agency_Name VARCHAR2(100),
  4   Email VARCHAR2(100),
  5   Phone VARCHAR2(15)
  6 )
  7 ;
```

Table created.

PackageAvailability Table

The PackageAvailability table stores the current availability information for each travel package.

```
SQL> CREATE TABLE PackageAvailability (
  2   Availability_ID NUMBER,
  3   Package_ID NUMBER,
  4   Available_Slots NUMBER
  5 );
```

Table created.

Establish Relationships

TravelAgency and TravelPackage

One travel agency can provide multiple travel packages. Therefore, the relationship is One-to-Many (1 to *). Agency_ID in TravelPackage is connected to Agency_ID in TravelAgency.

TravelPackage and PackageAvailability

Each travel package has one availability record. Therefore, the relationship is One-to-One (1 to 1). Package_ID in PackageAvailability refers to Package_ID in TravelPackage.

Primary Key Constraints

A Primary Key uniquely identifies each record in a table.

```
SQL> ALTER TABLE PackageAvailability
  2  ADD CONSTRAINT pk_packageavailability PRIMARY KEY (Availability_ID);

Table altered.
```

Foreign Key Constraints

The Agency_ID foreign key connects each travel package to an existing travel agency

```
SQL> ALTER TABLE TravelPackage
  2  ADD CONSTRAINT FK_AGENCY
  3  FOREIGN KEY (Agency_ID)
  4  REFERENCES TravelAgency(Agency_ID);

Table altered.
```

Constraints

Unique Email ID

The UNIQUE constraint prevents duplicate email addresses for travel agencies

```
SQL> ALTER TABLE PackageAvailability
  2  ADD CONSTRAINT UQ_PACKAGE
  3  UNIQUE (Package_ID);

Table altered.
```

```
SQL> ALTER TABLE TravelAgency
  2  ADD CONSTRAINT UQ_AGENCY_EMAIL
  3  UNIQUE (Email);

Table altered.
```

Email NOT NULL

The NOT NULL constraint ensures that every agency has an email address.

```
SQL> ALTER TABLE TravelAgency
  2  MODIFY Email VARCHAR2(100) NOT NULL;

Table altered.
```

Check Available Slots

The CHECK constraint ensures that available slots cannot be negative

```
SQL> ALTER TABLE PackageAvailability
  2  ADD CONSTRAINT CHK_SLOTS
  3  CHECK (Available_Slots >= 0);

Table altered.
```

Check Agency ID

This constraint ensures that agency IDs are positive

```
SQL> ALTER TABLE TravelAgency
  2  ADD CONSTRAINT chk_agency_id
  3  CHECK (Agency_ID > 0);

Table altered.
```

Insert Values into TravelAgency Table

The INSERT INTO command adds agency records. These Agency_ID values match the agencies used by the Week 2 TravelPackage records.

```
SQL> SELECT * FROM TravelAgency;
```

```
AGENCY_ID
-----
AGENCY_NAME
-----
EMAIL
-----
PHONE
-----
          201
Tamil Nadu Travels
contact@tamilnadutravels.com
9876543210
```

```
AGENCY_ID
-----
AGENCY_NAME
-----
EMAIL
-----
PHONE
-----
          202
Global Holiday Tours
info@globalholidaytours.com
9876543211
```

```
AGENCY_ID
-----
AGENCY_NAME
-----
EMAIL
-----
PHONE
-----
          203
Adventure World Travels
hello@adventureworld.com
9876543212
```

```

AGENCY_ID
-----
AGENCY_NAME
-----
EMAIL
-----
PHONE
-----
      204
Dream Honeymoon Tours
booking@dreamhoneymoon.com
9876543213

AGENCY_ID
-----
AGENCY_NAME
-----
EMAIL
-----
PHONE
-----
      205
Family Trip Planners
support@familytripplanners.com
9876543214

AGENCY_ID
-----
AGENCY_NAME
-----
EMAIL
-----
PHONE
-----
      206
Spiritual Journey Tours
care@spiritualjourneys.com
9876543215

6 rows selected.

```

Insert Values into PackageAvailability Table

Each travel package receives one availability record containing the number of slots currently available for booking.

```
SQL> INSERT INTO PackageAvailability  
2 VALUES (1, 101, 20);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PackageAvailability  
2 VALUES (2, 102, 15);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PackageAvailability  
2 VALUES (3, 103, 10);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PackageAvailability  
2 VALUES (4, 104, 12);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PackageAvailability  
2 VALUES (5, 105, 18);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PackageAvailability  
2 VALUES (6, 106, 14);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PackageAvailability  
2 VALUES (7, 107, 10);
```

1 row created.

```
SQL>  
SQL> INSERT INTO PackageAvailability  
2 VALUES (8, 108, 8);
```

1 row created.

```

SQL>
SQL> INSERT INTO PackageAvailability
  2 VALUES (9, 109, 20);

1 row created.

SQL>
SQL> INSERT INTO PackageAvailability
  2 VALUES (10, 110, 16);

1 row created.

SQL>
SQL> INSERT INTO PackageAvailability
  2 VALUES (11, 111, 25);

1 row created.

SQL>
SQL> INSERT INTO PackageAvailability
  2 VALUES (12, 112, 20);

1 row created.

```

```

SQL> SELECT * FROM PackageAvailability;

AVAILABILITY_ID PACKAGE_ID AVAILABLE_SLOTS
-----
          1         101             20
          2         102             15
          3         103             10
          4         104             12
          5         105             18
          6         106             14
          7         107             10
          8         108              8
          9         109             20
         10         110             16
         11         111             25

AVAILABILITY_ID PACKAGE_ID AVAILABLE_SLOTS
-----
          12         112             20

12 rows selected.

```

Report 1: Agency-wise Travel Package Report

This report displays the agency name, number of packages offered, and total available slots. JOIN operations combine TravelAgency, TravelPackage, and PackageAvailability.

```
SQL> SELECT a.Agency_Name,
2         COUNT(p.Package_ID) AS Number_of_Packages,
3         SUM(pa.Available_Slots) AS Available_Slots
4 FROM TravelAgency a
5 JOIN TravelPackage p
6 ON a.Agency_ID = p.Agency_ID
7 JOIN PackageAvailability pa
8 ON p.Package_ID = pa.Package_ID
9 GROUP BY a.Agency_Name
10 ORDER BY a.Agency_Name;
```

AGENCY_NAME		

NUMBER_OF_PACKAGES	AVAILABLE_SLOTS	

Adventure World Travels	2	32
Dream Honeymoon Tours	2	18
Family Trip Planners	2	36

AGENCY_NAME		

NUMBER_OF_PACKAGES	AVAILABLE_SLOTS	
-----	-----	
Global Holiday Tours	2	22
Spiritual Journey Tours	2	45
Tamil Nadu Travels	1	20

6 rows selected.

COUNT() calculates the number of packages supplied by each agency, while SUM() calculates their total available slots

Report 2: Package Availability Report

This report displays the package name, available slots, and booking status. CASE is used to identify whether a package is available or fully booked.

6 rows selected.

```
SQL> SELECT p.Package_Name,  
2         pa.Available_Slots,  
3         CASE  
4             WHEN pa.Available_Slots = 0  
5                 THEN 'Fully Booked'  
6                 ELSE 'Available'  
7         END AS Stock_Status  
8 FROM TravelPackage p  
9 JOIN PackageAvailability pa  
10 ON p.Package_ID = pa.Package_ID  
11 ORDER BY pa.Available_Slots;
```

PACKAGE_NAME

AVAILABLE_SLOTS STOCK_STATUS

Kashmir Honeymoon Package
8 Available

Dubai Holiday Package
10 Available

Goa Honeymoon Package
10 Available

PACKAGE_NAME

AVAILABLE_SLOTS STOCK_STATUS

Singapore Tour Package
12 Available

Rishikesh Adventure Tour
14 Available

Rajasthan Family Tour
16 Available

PACKAGE_NAME

AVAILABLE_SLOTS STOCK_STATUS

Manali Adventure Trip
18 Available

Varanasi Spiritual Tour
20 Available

Ooty Holiday Package
20 Available

PACKAGE_NAME

AVAILABLE_SLOTS STOCK_STATUS

Mysore Family Package
20 Available

Tirupati Pilgrimage Tour
25 Available

11 rows selected.

Report 3: Availability Analysis A. Total Packages Available

```
SQL> SELECT COUNT(*) AS Total_Packages_Available
 2  FROM PackageAvailability
 3  WHERE Available_Slots > 0;

TOTAL_PACKAGES_AVAILABLE
-----
12
```

This counts packages having at least one available slot.

B. Packages Out of Stock

```
SQL> SELECT COUNT(*) AS Packages_Out_of_Stock
 2  FROM PackageAvailability
 3  WHERE Available_Slots = 0;

PACKAGES_OUT_OF_STOCK
-----
0
```

This counts packages with no available slots.

C. Highest-Stocked Packages

```
SQL> SELECT p.Package_Name,
 2         pa.Available_Slots
 3  FROM TravelPackage p
 4  JOIN PackageAvailability pa
 5  ON p.Package_ID = pa.Package_ID
 6  WHERE pa.Available_Slots = (
 7         SELECT MAX(Available_Slots)
 8         FROM PackageAvailability
 9  );

PACKAGE_NAME
-----
AVAILABLE_SLOTS
-----
Tirupati Pilgrimage Tour
25
```

MAX() finds the highest availability, and the query displays the package or packages having that value

D. Average Availability Quantity

```
SQL> SELECT AVG(Available_Slots) AS Average_Availability
 2  FROM PackageAvailability;

AVERAGE_AVAILABILITY
-----
15.6666667
```